MTH8302 - Devoir 1 : Rappel Mathématique

February 9, 2026

1. **Problème 1 : Gradients et Hessiennes** ***(15 points)***

   - **Rappel :**
     - Une matrice $\mathbf{A} \in \mathbb{R}^{n\times n}$ est symétrique si $\mathbf{A}^{\mathrm{T}} = \mathbf{A}$.
     - Le gradient $\nabla f(x)$ d'une fonction $f : \mathbb{R}^n \to \mathbb{R}$ est un vecteur contenant les dérivées partielles tel que :
     $$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{bmatrix} = \left(\frac{\partial f}{\partial \mathbf{x}}\right).$$
     - La Hessienne $\nabla^2 f(\mathbf{x})$ est une matrice symétrique $n \times n$ contenant les dérivées secondes :
     $$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$
     - **Règle de la dérivée en chaîne (matricielle) :** Pour une fonction composée $f(h(\mathbf{x}))$, la dérivée est donnée par :
     $$\frac{\partial f}{\partial \mathbf{x}} = \frac{\partial f}{\partial h} \cdot \frac{\partial h}{\partial \mathbf{x}}.$$
     - **Règle du produit matriciel :** Pour deux matrices différentiables $\mathbf{A}(\mathbf{x})$ et $\mathbf{B}(\mathbf{x})$, la dérivée est :
     $$\frac{\partial}{\partial \mathbf{x}}(\mathbf{AB}) = \frac{\partial \mathbf{A}}{\partial \mathbf{x}}\mathbf{B} + \mathbf{A}\frac{\partial \mathbf{B}}{\partial \mathbf{x}}.$$
     - **Dérivée d'un produit de type vecteur-matrice-vecteur :** Si $\mathbf{b}$ et $\mathbf{x}$ sont des vecteurs et $\mathbf{A}$ est une matrice, alors la dérivée de $\mathbf{b}^{\mathrm{T}}\mathbf{Ax}$ par rapport à $\mathbf{x}$ est :
     $$\frac{\partial}{\partial \mathbf{x}}(\mathbf{b}^{\mathrm{T}}\mathbf{Ax}) = \mathbf{A}^{\mathrm{T}}\mathbf{b}.$$
     - **Dérivée d'une forme quadratique :** Si $\mathbf{A} \in \mathbb{R}^{n\times n}$ n'est pas nécessairement symétrique, alors la dérivée de $\mathbf{x}^{\mathrm{T}}\mathbf{Ax}$ est :
     $$\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^{\mathrm{T}}\mathbf{Ax}) = (\mathbf{A} + \mathbf{A}^{\mathrm{T}})\,\mathbf{x}.$$
     - Pour justifier votre réponse Vous pouvez suivre les étapes suivantes :

    * 1. Expansion de l'expression en termes de scalaires.
    * 2. Calcul de la dérivée de l'expression en termes de scalaires.
    * 3. Réécriture de la dérivée calculée en notation matricielle ou vectorielle.
  - Vous pouvez aussi utiliser les règles de dérivées partielles comme justification (Vous pouvez en trouver ici *The Matrix Cookbook*.)
  - Toute autre méthode de justification valide est acceptée.

- **Problème 1 : Question 1**
  - Soit $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x} + \mathbf{b}^{\mathrm{T}}\mathbf{x} + c$, où :
    * $\mathbf{A} \in \mathbb{R}^{n\times n}$ est une matrice symétrique.
    * $\mathbf{x} \in \mathbb{R}^{n}$ est un vecteur réel.
    * $\mathbf{b} \in \mathbb{R}^{n}$ est un vecteur réel.
    * $c \in \mathbb{R}$ un scalaire réel.
  - Trouvez $\nabla_{\mathbf{x}} f(\mathbf{x})$ en choisissant la bonne réponse parmi les choix suivants :
    (a) $\nabla_{\mathbf{x}} f(\mathbf{x}) = \frac{1}{2}\mathbf{A}\mathbf{x} + \mathbf{b}$
    (b) $\nabla_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$
    (c) $\nabla_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}\mathbf{x} + \mathbf{b}$
    (d) $\nabla_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$
  - Justifiez votre réponse ***(2.5 points)***.

- **Problème 1 : Question 2**
  - Soit $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x} + \mathbf{b}^{\mathrm{T}}\mathbf{x} + c$, où :
    * $\mathbf{A} \in \mathbb{R}^{n\times n}$ est une matrice symétrique.
    * $\mathbf{x} \in \mathbb{R}^{n}$ est un vecteur réel.
    * $\mathbf{b} \in \mathbb{R}^{n}$ est un vecteur réel.
    * $c \in \mathbb{R}$ un scalaire réel.
  - Trouvez $\nabla^2_{\mathbf{x}} f(\mathbf{x})$ en choisissant la bonne réponse parmi les choix suivants :
    (a) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}$
    (b) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \frac{1}{2}\mathbf{A}$
    (c) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A} + \mathbf{A}^{\mathrm{T}}$
    (d) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}^{\mathrm{T}}\mathbf{A}$
  - Justifiez votre réponse ***(2.5 points)***.

- **Problème 1 : Question 3**
  - Soit $f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}$, où :
    * $\mathbf{A} \in \mathbb{R}^{n\times n}$ est une matrice symétrique.
  - Trouvez $\nabla_{\mathbf{x}} f(\mathbf{x})$ en choisissant la bonne réponse parmi les choix suivants :
    (a) $\nabla f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}\mathbf{A}\mathbf{x}$
    (b) $\nabla f(\mathbf{x}) = 2e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}\mathbf{A}\mathbf{x}$
    (c) $\nabla f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}\mathbf{A}^{\mathrm{T}}\mathbf{x}$
    (d) $\nabla f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}(\mathbf{A}\mathbf{x} + \mathbf{A}^{\mathrm{T}}\mathbf{x})$
  - Justifiez votre réponse (Vous pouvez utiliser la règle de la dérivée en chaîne) ***(2.5 points)***.

- **Problème 1 : Question 4**
  - Soit $f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}$, où :

    * $\mathbf{A} \in \mathbb{R}^{n\times n}$ est une matrice symétrique.
  - Trouvez $\nabla^2_{\mathbf{x}} f(\mathbf{x})$ en choisissant la bonne réponse parmi les choix suivants :
    (a) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}(2\mathbf{A} + 4\mathbf{A}\mathbf{x}\mathbf{x}^{\mathrm{T}}\mathbf{A})$
    (b) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}(\mathbf{A} + 4\mathbf{A}\mathbf{x}\mathbf{x}^{\mathrm{T}}\mathbf{A})$
    (c) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}(2\mathbf{A} + 2\mathbf{A}^{\mathrm{T}})$
    (d) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = e^{\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x}}(2\mathbf{A} + 2\mathbf{A}\mathbf{x}\mathbf{x}^{\mathrm{T}}\mathbf{A})$
  - Justifiez votre réponse (Vous pouvez utiliser la règle de la dérivée du produit matriciel) ***(2.5 points)***.

- **Problème 1 : Question 5**
  - Soit $f(\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 = (\mathbf{A}\mathbf{x})^{\mathrm{T}}(\mathbf{A}\mathbf{x})$, où :
    * $A \in \mathbb{R}^{m\times n}$ est une matrice donnée.
    * $\mathbf{x} \in \mathbb{R}^n$ est un vecteur réel composé de n éléments.
  - Trouvez $\nabla_{\mathbf{x}} f(\mathbf{x})$ en choisissant la bonne réponse parmi les choix suivants :
    (a) $\nabla_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}^{\mathrm{T}}\mathbf{A}\mathbf{x}$
    (b) $\nabla_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}^{\mathrm{T}}\mathbf{A}\mathbf{x}$
    (c) $\nabla_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}\mathbf{A}^{\mathrm{T}}\mathbf{x}$
    (d) $\nabla_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}\mathbf{A}^{\mathrm{T}}\mathbf{x}$
  - Justifiez votre réponse ***(2.5 points)***.

- **Problème 1 : Question 6**
  - Soit $f(\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 = (\mathbf{A}\mathbf{x})^{\mathrm{T}}(\mathbf{A}\mathbf{x})$, où :
    * $A \in \mathbb{R}^{m\times n}$ est une matrice donnée.
    * $\mathbf{x} \in \mathbb{R}^n$ est un vecteur réel composé de n éléments.
  - Trouvez $\nabla^2_{\mathbf{x}} f(\mathbf{x})$ en choisissant la bonne réponse parmi les choix suivants :
    (a) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}^{\mathrm{T}}\mathbf{A}$
    (b) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}^{\mathrm{T}}\mathbf{A}$
    (c) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = 2\mathbf{A}\mathbf{A}^{\mathrm{T}}$
    (d) $\nabla^2_{\mathbf{x}} f(\mathbf{x}) = \mathbf{A}\mathbf{A}^{\mathrm{T}}$
  - Justifiez votre réponse ***(2.5 points)***.

2. **Problème 2 - Optimisation : Descente de gradient vs Newton et analyse spectrale *(35 points)***

- On considère la fonction (non quadratique) définie sur le domaine $x_1 > 0$ et $x_2 > 0$:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top\mathbf{A}\mathbf{x} - \mathbf{b}^\top\mathbf{x} - \mu\big(\log(x_1) + \log(x_2)\big), \qquad \mathbf{x} = (x_1, x_2)^\top \in \mathbb{R}^2.$$

- Les paramètres sont fixés à

$$\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \mu = 0.1, \qquad \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

- On compare les deux méthodes suivantes :
  - **Descente de gradient (pas constant)** :

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta\,\nabla f(\mathbf{x}_k), \qquad \eta_0 = 0.05.$$

– **Méthode de Newton (second ordre)** : on calcule $\mathbf{p}_k$ en résolvant

$$\mathbf{H}(\mathbf{x}_k)\mathbf{p}_k = -\nabla f(\mathbf{x}_k), \qquad \mathbf{H}(\mathbf{x}) = \nabla^2 f(\mathbf{x}),$$

puis on met à jour $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha\,\mathbf{p}_k$, avec $\alpha \in (0, 1]$.

- Critère d'arrêt : $\|\nabla f(\mathbf{x}_k)\| < 10^{-6}$ (ou un nombre maximal d'itérations).
- **Attention :** la fonction f n'est définie que pour $x_1 > 0$ et $x_2 > 0$. Si une mise à jour sort du domaine, vous devez appliquer un **backtracking simple** : on divise le pas par 2 et on réessaye jusqu'à obtenir un point strictement positif.

(a) Calculer l'expression correcte pour $\nabla f(\mathbf{x})$ en choisissant le bon choix parmi les choix suivants et en justifiant votre réponse ***(2.5 points)***:

i. $\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b} - \mu \begin{bmatrix} 1/x_1 \\ 1/x_2 \end{bmatrix}$

ii. $\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} - \mu \begin{bmatrix} 1/x_1 \\ 1/x_2 \end{bmatrix}$

iii. $\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} + \mu \begin{bmatrix} 1/x_1 \\ 1/x_2 \end{bmatrix}$

iv. $\nabla f(\mathbf{x}) = \mathbf{A}^\top\mathbf{x} - \mathbf{b} - \mu \begin{bmatrix} 1/x_1 \\ 1/x_2 \end{bmatrix}$ (car A n'est pas symétrique)

(b) Trouver l'expression correcte pour la Hessienne $\mathbf{H}(\mathbf{x}) = \nabla^2 f(\mathbf{x})$ en choisissant le bon choix parmi les choix suivants et en justifiant votre réponse ***(2.5 points)***:

i. $\mathbf{H}(\mathbf{x}) = \mathbf{A} - \mu\,\mathrm{diag}\left(\frac{1}{x_1^2}, \frac{1}{x_2^2}\right)$

ii. $\mathbf{H}(\mathbf{x}) = \mathbf{A} + \mu\,\mathrm{diag}\left(\frac{1}{x_1^2}, \frac{1}{x_2^2}\right)$

iii. $\mathbf{H}(\mathbf{x}) = \mathbf{A} + \mu \begin{bmatrix} 0 & 1/(x_1x_2) \\ 1/(x_1x_2) & 0 \end{bmatrix}$

iv. $\mathbf{H}(\mathbf{x}) = \mathbf{A} + \mu \begin{bmatrix} 1/x_1 & 0 \\ 0 & 1/x_2 \end{bmatrix}$

(c) Quand $x_1 \to 0^+$ (ou $x_2 \to 0^+$), quelles affirmations sont vraies? Justifier votre choix ***(2.5 points)***.

i. $-\mu \log(x_1) \to +\infty$, donc $f(\mathbf{x}) \to +\infty$.

ii. Le gradient contient un terme $-\mu/x_1$ qui devient très grand en magnitude.

iii. La courbure locale augmente car la Hessienne contient μ/x_1^2.

iv. a, b et c.

(d) On considère le modèle local quadratique :

$$f(\mathbf{x}_k + \mathbf{p}) \approx f(\mathbf{x}_k) + \mathbf{g}^\top\mathbf{p} + \frac{1}{2}\,\mathbf{p}^\top\mathbf{H}\mathbf{p}, \quad \mathbf{g} = \nabla f(\mathbf{x}_k),\ \mathbf{H} = \nabla^2 f(\mathbf{x}_k).$$

À quelle équation la minimisation de ce modèle conduit-elle? Choisir la bonne réponse parmi les choix suivants en justifiant votre choix ***(2.5 points)***:

i. $\mathbf{H}\mathbf{p} = \mathbf{g}$

ii. $\mathbf{H}\mathbf{p} = -\mathbf{g}$

iii. $\mathbf{p} = -\eta\,\mathbf{g}$

iv. $\mathbf{p} = \mathbf{H}^{-1}\mathbf{g}$ (inverse explicite obligatoire)

(e) **Implémentation Python *(16 points)*.** La fonction f n'est définie que si $x_1 > 0$ et $x_2 > 0$. Si une mise à jour sort du domaine, il faudrait alors mettre en place un backtracking simple : diviser le pas par 2 et réessayer. Pour la descente de gradient et pour Newton, on peut appliquer un facteur $\alpha \in (0, 1]$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha\, \mathbf{p}_k,$$

et réduire α si nécessaire.

i. compléter le code Python qui calcule $f(\mathbf{x})$, $\nabla f(\mathbf{x})$, $\mathbf{H}(\mathbf{x})$ et implémente le backtracking simple ***(8 points)*** .

```
import numpy as np
A = np.array([[4.0, 1.0],
              [1.0, 3.0]])
b = np.array([1.0, 2.0])
mu = 0.1
def f(x):
    x1, x2 = x[0], x[1]
    ____
    return ____
def grad_f(x):
    ____
return ____
def H_f(x):
    ____
    return ____

def backtracking_to_stay_positive(x, step, alpha0=1.0):
alpha = ____
while np.any(x + ____ <= ____):
    alpha *= ____
return alpha
```

ii. Compléter le code Python qui exécute la descente de gradient ainsi que la méthode de Newton à partir de $\mathbf{x}_0$ tout en utilisant la méthode simple de backtracking. On voudrait conserver l'historique des itérés $\{\mathbf{x}_k\}$ pour des fins de visualisation ***(8 points)***.

.

```
def run_gd(x0, eta=0.05, tol=1e-6, max_iter=2000):
    x = x0.astype(float).copy()
    hist = [x.copy()]
    for k in range(max_iter):
        g = grad_f(x)
        if np.linalg.norm(g) < tol:
            break
        step = ____
        alpha = ___
        x = ____
        hist.____(x.____)
    return np.array(____)
def run_newton(x0, tol=1e-6, max_iter=100):
    x = x0.astype(float).copy()
    hist = [x.copy()]
    for k in range(max_iter):
        g = grad_f(x)
        if np.linalg.norm(g) < tol:
            break
        H = ____
        p = ____
        alpha = ____
        x = ____
        hist.____(x.____)
    return np.array(____)
```

(f) **Visualisation et interprétation *(4 points)*.**

- Compléter et exécuter le code ici pour tracer ***(2 points)***

- (i) les courbes de niveau de f sur une grille $x_1 \in [0.1, 3]$, $x_2 \in [0.1, 3]$,
- (ii) les trajectoires des itérés pour GD et Newton (sur la même figure),
- (iii) l'évolution de $\|\nabla f(\mathbf{x}_k)\|$ en fonction de k pour chaque méthode.

,

- Quelle méthode converge le plus vite (en nombre d'itérations)? Pourquoi? ***(2 points)***

.

```
import matplotlib.pyplot as plt
x0 = np.array([1.0, 1.0])
hist_gd = ____(____, eta=0.05)
hist_nt = ____(____)
# Grille pour contours
x1 = np.linspace(0.1, 3.0, 200)
x2 = np.linspace(0.1, 3.0, 200)
X1, X2 = np.meshgrid(x1, x2)
Z = np.zeros_like(X1)
for i in range(X1.shape[0]):
    for j in range(X1.shape[1]):
        Z[i, j] = f(np.array([X1[i, j], X2[i, j]]))
plt.figure(figsize=(7,6))
plt.contour(X1, X2, Z, levels=25)
plt.plot(hist_gd[:,0], hist_gd[:,1], marker='o', markersize=2, label='GD')
plt.plot(hist_nt[:,0], hist_nt[:,1], marker='o', markersize=2, label='Newton')
plt.scatter([x0[0]],[x0[1]], s=60, label='x0')
plt.xlabel(r"$x_1$")
plt.ylabel(r"$x_2$")
plt.legend()
plt.title("Courbes de niveau et trajectoires")
plt.show()

# Norme du gradient
plt.figure(figsize=(7,4))
plt.plot([np.linalg.norm(grad_f(x)) for x in hist_gd], label='GD')
plt.plot([np.linalg.norm(grad_f(x)) for x in hist_nt], label='Newton')
plt.yscale("log")
plt.xlabel("itération k")
plt.ylabel(r"$\|\nabla f(x_k)\|$")
plt.legend()
plt.title("Évolution de la norme du gradient")
plt.show()
```

(g) **Analyse spectrale locale** ***(5 points)***.
On définit

$$\mathbf{x}^{(1)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \mathbf{x}^{(2)} = \begin{bmatrix} 0.2 \\ 0.2 \end{bmatrix}.$$

i. Compléter le code ci-dessous pour calculer les valeurs propres et les vecteurs propres de $\mathbf{H}(\mathbf{x}^{(1)})$ et $\mathbf{H}(\mathbf{x}^{(2)})$. Reportez dans votre rapport $\lambda_{\max}$, $\lambda_{\min}$ et les vecteurs propres associés ***(2 points)***.

ii. Le nombre de conditionnement de $\mathbf{H}$ est donné par :

$$\kappa(\mathbf{x}) = \frac{\lambda_{\max}(\mathbf{H}(\mathbf{x}))}{\lambda_{\min}(\mathbf{H}(\mathbf{x}))},$$

Que represente $\kappa(\mathbf{x})$ et qu'est-ce qu'il indique ***(1 point)***?

iii. calculer $\kappa(\mathbf{x}^{(1)})$ et $\kappa(\mathbf{x}^{(2)})$ et déduire le comportement de la descente de gradient aux points $\mathbf{x}^{(1)}$ et $\mathbf{x}^{(2)}$ ***(2 points)***.

.

```
def spectral_analysis(H: np.ndarray):
    """
    Retourne (lambda_min, v_min, lambda_max, v_max, eigvals_sorted, eigvecs_sorted)
```

```
    pour une matrice symétrique 2x2.
    """
    eigvals, eigvecs = ____  # pour matrices symétriques : valeurs propres triées croissant
    lam_min, lam_max = eigvals[0], eigvals[-1]
    v_min, v_max = eigvecs[:, 0], eigvecs[:, -1]  # colonnes = vecteurs propres associés
    return lam_min, v_min, lam_max, v_max, eigvals, eigvecs
# Points demandés
x1 = np.array(____)    # loin de la frontière
x2 = np.array(____)    # proche de la frontière
for name, x in [("x^(1) = (1,1)", x1), ("x^(2) = (0.2,0.2)", x2)]:
    H = H_f(x)
    lam_min, v_min, lam_max, v_max, eigvals, eigvecs = ____
    print("=" * 70)
    print(f"Point {name}")
    print("H(x) =\n", H)
    print("\nValeurs propres (triées) :")
    print("eigvals =", eigvals)
    print("\nVecteurs propres (colonnes, associés aux valeurs propres ci-dessus) :")
    print("eigvecs =\n", eigvecs)
    print("\nIdentification :")
    print(f"lambda_min = {lam_min:.6f},  vecteur propre associé v_min = {v_min}")
    print(f"lambda_max = {lam_max:.6f},  vecteur propre associé v_max = {v_max}")
```

3. **Problème 3 : Estimation par la Méthode du Maximum de Vraisemblance et Propriétés des Estimateurs** ***(25 points)***

   - Soit $X_1, X_2, \ldots, X_n$ un échantillon aléatoire issu d'une **loi de Poisson** de paramètre λ, c'est-à-dire :
   $$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \ldots$$

   (a) Déterminer l'estimateur du maximum de vraisemblance (EMV) du paramètre λ ***(5 points)***.

   (b) Vérifier si cet estimateur est biaisé ***(5 points)***.

   (c) Vérifier si cet estimateur est consistant (ou convergent) en calculant sa variance ***(5 points)***. On appelle **erreur quadratique moyenne** la quantité :
   $$EQM(\hat{\theta}_n) = \mathbb{E}\left[(\hat{\theta}_n - \theta)^2\right] = V(\hat{\theta}_n) + [b(\hat{\theta}_n)]^2$$
   Un estimateur est dit **consistant** si :
   $$\lim_{n\to\infty} EQM(\hat{\theta}_n) = 0$$
   .

   (d) On observe les fréquences suivantes pour une variable X suivant une loi de Poisson :

x	$f_X(x)$
0	7
1	14
2	12
3	13
4	6
5	3

   i. Trouver l'estimateur du maximum de vraisemblance (EMV) de λ en utilisant les données observées ***(5 points)***.

   ii. Estimer la probabilité $P(X = 2)$ ***(5 points)***.

4. **Problème 4 : Test F de Fisher et Intervalle de Confiance** ***(20 points)***

- Une université teste une nouvelle méthode d'étude et souhaite comparer la variabilité des scores aux examens. Deux groupes d'étudiants ont passé le même examen :
  - **Groupe A (méthode classique) :** $[72, 75, 78, 69, 82, 77, 71, 74, 80, 73]$.
  - **Groupe B (nouvelle méthode) :** $[78, 81, 85, 79, 88, 84, 82, 80, 86, 83]$.
- Taille des échantillons : $n_A = n_B = 10$.
  (a) Effectuer un **test F de Fisher** pour comparer les variances des deux groupes pour un niveau de signification $\alpha = 0.05$ ***(5 points)***.
  (b) Tracer la courbe de la distribution F montrant la région critique (zone de rejet de H_0) et la statistique F_{stat} observée ***(5 points)***.
  (c) Calculer un intervalle de confiance pour le ratio des variances à 95% ***(5 points)***.
  (d) Fournir le code source pour le calcul de l'intervalle de confiance et interpréter le résultat obtenu (Est-ce que la valeur d'un ratio égale à 1 est dans l'intervalle?) ***(5 points)***.

5. **Problème 5 : Loi des Grands Nombres et Théorème Central Limite *(5 points)***

- On simule une variable aléatoire de Bernoulli ($p = 0.5$) et on calcule la moyenne empirique sur n observations. On vous fournit le code et on vous demande de modifier certains paramètres avant d'exécuter to observer les résultats.
- **Problème 5 - Question 1 :** Dans le code suivant, modifiez la valeur de $n = [50, 500, 5000]$. Quelle est l'influence de n sur la dispersion de $\overline{X}_n$ ***(2 points)***?

.

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme()

np.random.seed(42)

# Paramètres
n = 100  # Nombre d'échantillons
num_trials = 2000  # Nombre de répétitions

# Génération des variables de Bernoulli
X = np.random.binomial(1, 0.5, (num_trials, n))

# Calcul des moyennes cumulatives
means = np.cumsum(X, axis=1) / np.arange(1, n + 1)

final_means = means[:, -1]  # Moyenne

plt.figure(figsize=(10, 5))
plt.hist(final_means, bins=20, density=True, alpha=0.6, color='b', edgecolor='black')
plt.axvline(0.5, color='red', linestyle='dashed', label=r'$\mathbb{E}[X] = 0.5$')
plt.title("Loi faible des grands nombres : distribution de $\overline{X}_n$")
plt.xlabel("Valeur de la moyenne empirique")
plt.ylabel("Densité")
plt.legend()
plt.show()
```

- **Problème 5 - Question 2 :** Pour une valeur fixe de $n = 1000$, Que se passe-t-il si l'on répète l'expérience plusieurs fois ou même une seule fois ($num_{\text{trials}} = [1, 5, 10, 1000]$)? Qu'est-ce qui est observé en commun dans tous les cas de figures ***(2 points)***?

.

```python
# LOI FORTE DES GRANDS NOMBRES: Trajectoires individuelles
# Paramètres
n = 10  # Nombre d'échantillons
num_trials = 5  # Nombre de répétitions

# Génération des variables de Bernoulli
X = np.random.binomial(1, 0.5, (num_trials, n))

# Calcul des moyennes cumulatives
means = np.cumsum(X, axis=1) / np.arange(1, n + 1)

plt.figure(figsize=(10, 5))
for i in range(5):  # Tracer 5 trajectoires différentes
    plt.plot(means[i, :], alpha=0.7, label=f'Trajectoire {i+1}')

plt.axhline(0.5, color='red', linestyle='dashed', label=r'$\mathbb{E}[X] = 0.5$')
plt.title("Loi forte des grands nombres : trajectoires de $\overline{X}_n$")
plt.xlabel("Nombre d'échantillons $n$")
plt.ylabel("Moyenne empirique $\overline{X}_n$")
plt.legend()
plt.show()
```

- **Problème 5 - Question 3 :** Changez les valeurs de n $(100, 1000, 10000)$ et observez comment la distribution évolue. Vers quelle densité l'histogramme converge ***(1 point)***?

.

```python
import scipy.stats as stats

num_trials = 500
for n in [100, 1000, 10000]:
    X_tcl = np.random.binomial(1, 0.5, (num_trials, n))
    sample_means = np.mean(X_tcl, axis=1)

    plt.figure(figsize=(10, 5))
    plt.hist(sample_means, bins=20, density=True, alpha=0.6, color='g', edgecolor='black', label="Moyennes empiriques")

    # Superposition d'une distribution normale
    mu, sigma = 0.5, np.sqrt(0.5 * 0.5 / n)
    x_vals = np.linspace(0.3, 0.7, 100)
    plt.plot(x_vals, stats.norm.pdf(x_vals, mu, sigma), color="red", lw=2, label="Densité normale")

    plt.axvline(0.5, color='black', linestyle='dashed', label=r'$\mathbb{E}[X] = 0.5$')
    plt.title(f"Théorème Central Limite : Distribution pour $n={n}$")
    plt.xlabel("Valeur de la moyenne empirique")
    plt.ylabel("Densité")
    plt.legend()
    plt.show()
```